



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра квантовых информационных технологий, практической и
прикладной информатики (ККИТПиПИ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ
по дисциплине «Анализ больших данных»
Практическая работа №2

Студент группы ИНБО-12-23, Албахтин И.В.

(подпись)

Преподаватель Шендяпин А.В.

(подпись)

Москва 2026 г.

Практическая работа №2. Визуализация данных в Python

1. Цель и задачи работы

Цель работы — освоить статическую и интерактивную визуализацию данных в Python, исследовать влияние параметров t-SNE и UMAP на двумерное представление многомерных объектов и сравнить измеренное время работы алгоритмов.

В работе выполнены следующие задачи:

- загрузка и диагностика многомерного набора Bike Sharing средствами Pandas;
- построение столбчатой и круговой диаграмм с помощью Plotly graph_objects;
- построение линейных графиков с помощью Matplotlib;
- визуализация набора рукописных цифр при разных значениях перплексии t-SNE;
- визуализация тех же объектов UMAP при разных `n_neighbors` и `min_dist`;
- измерение времени и сравнение возможностей библиотек.

2. Исходные данные и первичная диагностика

Основной набор — дневная таблица Capital Bikeshare за 2011–2012 годы. Она содержит 731 наблюдение и 16 исходных столбцов: календарные признаки, сезон, погодные условия, нормированные температуру, влажность и скорость ветра, а также число поездок случайных и зарегистрированных пользователей. Источник: UCI Machine Learning Repository, Bike Sharing Dataset, автор Hadi Fanaee-T.

Таблица 1 – Основные группы признаков Bike Sharing

Группа	Поля	Смысл
Идентификатор и дата	instant, dteday	Номер записи и календарная дата
Календарь	season, yr, mnth, holiday, weekday, workingday	Сезон, год, месяц и тип дня

Группа	Поля	Смысл
Погода	weathersit, temp, atemp, hum, windspeed	Погодный класс и нормированные измерения
Спрос	casual, registered, cnt	Поездки по типам пользователей и итог

```
df = pd.read_csv("data/day.csv", parse_dates=["dteday"])
df.head()
df.info()
df.isna().sum()
df.duplicated().sum()
```

Таблица 2 – Первые пять строк исходного набора

Дата	Сезон	Погода	temp	hum	casual	registered	cnt
2011-01-01	1	2	0.344	0.806	331	654	985
2011-01-02	1	2	0.363	0.696	131	670	801
2011-01-03	1	1	0.196	0.437	120	1229	1349
2011-01-04	1	1	0.200	0.590	108	1454	1562
2011-01-05	1	1	0.227	0.437	82	1518	1600

Таблица 3 – Результат первичной диагностики

Показатель	Значение
Размер	731 строк × 16 столбцов
Период	2011-01-01 — 2012-12-31
Пропущенные значения	0
Полные дубликаты	0
Всего поездок	3 292 679
Среднее в день	4504.35
Медиана в день	4548
Максимум в день	8714

Таблица не содержит пропусков и полных дубликатов, поэтому удаление или интерполяция строк не требуются. Проверка casual + registered = cnt выполнена для всех 731 наблюдения и не выявила расхождений.

3. Предобработка данных

Дата преобразована в тип datetime. Числовые коды сезонов и погоды дополнены читаемыми категориальными названиями. Нормированные

погодные показатели переведены в исходные единицы согласно описанию набора.

```
clean["temp_c"] = clean["temp"] * 41
clean["atemp_c"] = clean["atemp"] * 50
clean["humidity_pct"] = clean["hum"] * 100
clean["windspeed_kmh"] = clean["windspeed"] * 67
assert (clean["casual"] + clean["registered"] == clean["cnt"]).all()
```

Очищенная таблица сохранена в CSV и Parquet. Исходные признаки не удаляются: производные поля добавлены отдельно, чтобы сохранить проверяемую связь с источником.

4. Столбчатая диаграмма Plotly

Дневные значения агрегированы по месяцам, чтобы даты оставались читаемыми. Высота и цвет столбца кодируют суммарное количество поездок за месяц. Граница каждого столбца чёрная, толщиной 2; высота интерактивного рисунка равна 700 px.

```
go.Bar(
    x=monthly["month_label"], y=monthly["cnt"],
    marker=dict(color=monthly["cnt"], coloraxis="coloraxis",
        line=dict(color="black", width=2))
)
fig.update_layout(
    title=dict(x=0.5, font=dict(size=20)), height=700,
    margin=dict(l=10, r=10, t=65, b=10)
)
fig.update_xaxes(title_font_size=16, tickfont_size=14,
    tickangle=315, gridwidth=2, gridcolor="ivory")
fig.update_yaxes(title_font_size=16, tickfont_size=14,
    gridwidth=2, gridcolor="ivory")
```

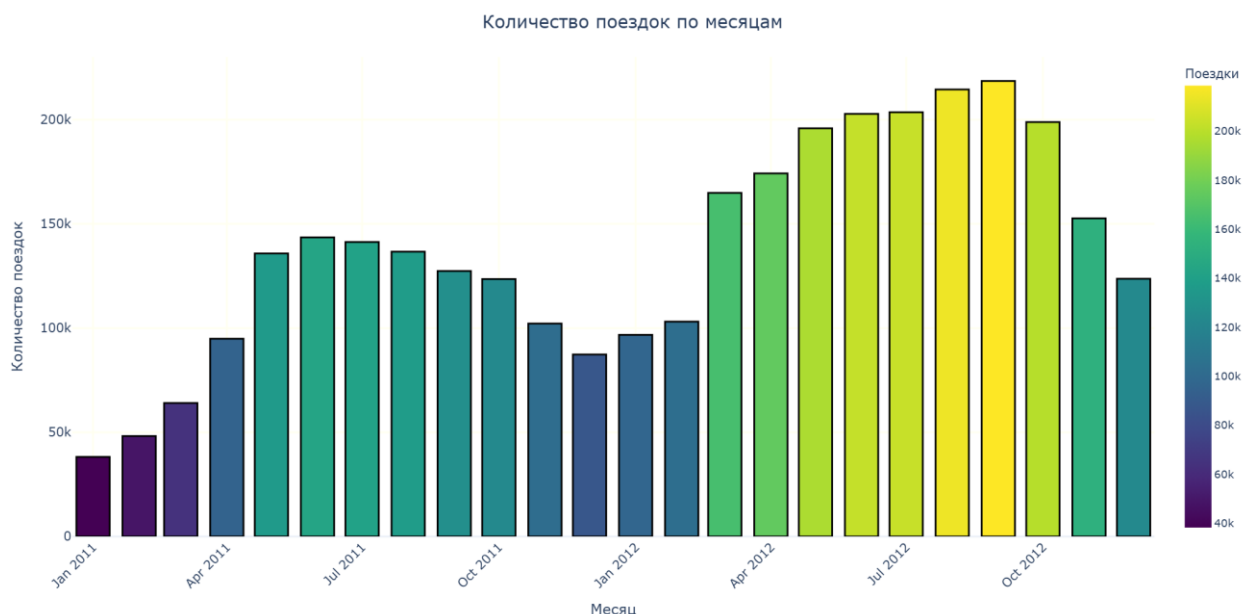


Рисунок 1 – Количество поездок по месяцам

Пиковый месяц — 2012-09: 218 573 поездок. В 2012 году столбцы в основном выше соответствующих месяцев 2011 года; внутри каждого года виден подъём от зимы к тёплому сезону и спад к концу года.

5. Круговая диаграмма Plotly

Для круговой диаграммы поездки сгруппированы по четырём сезонам. Небольшое число категорий позволяет разместить названия и доли без наложений.

```
go.Pie(
  labels=season_totals.index,
  values=season_totals.values,
  textinfo="label+percent",
  marker=dict(colors=pie_colors,
    line=dict(color="black", width=2))
)
```

Распределение поездок по сезонам

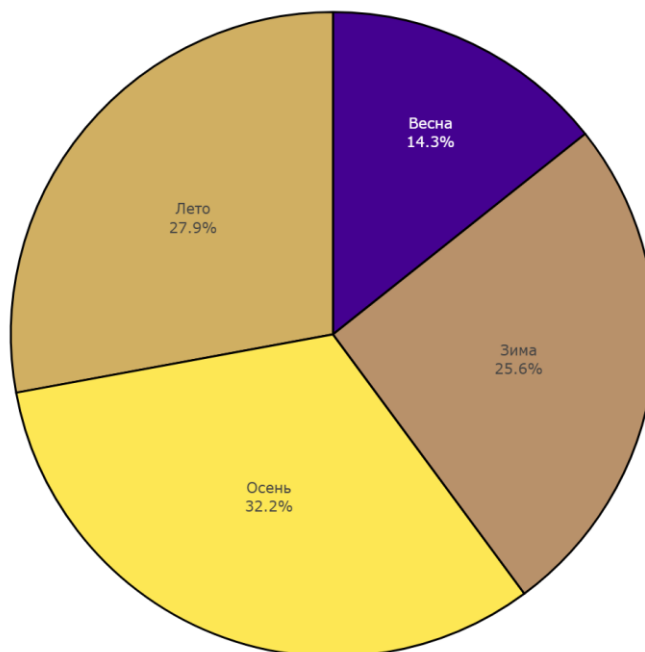


Рисунок 2 – Распределение поездок по сезонам

Осень формирует наибольший объём — 1 061 129 поездок (32.2%) а весна — наименьший: 471 348 (14.3%).

6. Линейные графики Matplotlib

По общей временной оси сопоставлены три месячных показателя: случайные, зарегистрированные и все поездки. Каждый ряд вынесен в отдельную область построения и оформлен линией crimson, белыми маркерами с чёрной границей толщиной 2 и сеткой mistyrose толщиной 2.

```
ax.plot(x, y, color="crimson", linewidth=2, marker="o",  
        markerfacecolor="white", markeredgewidth=2,  
        markeredgewidth=2)  
ax.grid(linewidth=2, color="mistyrose")
```

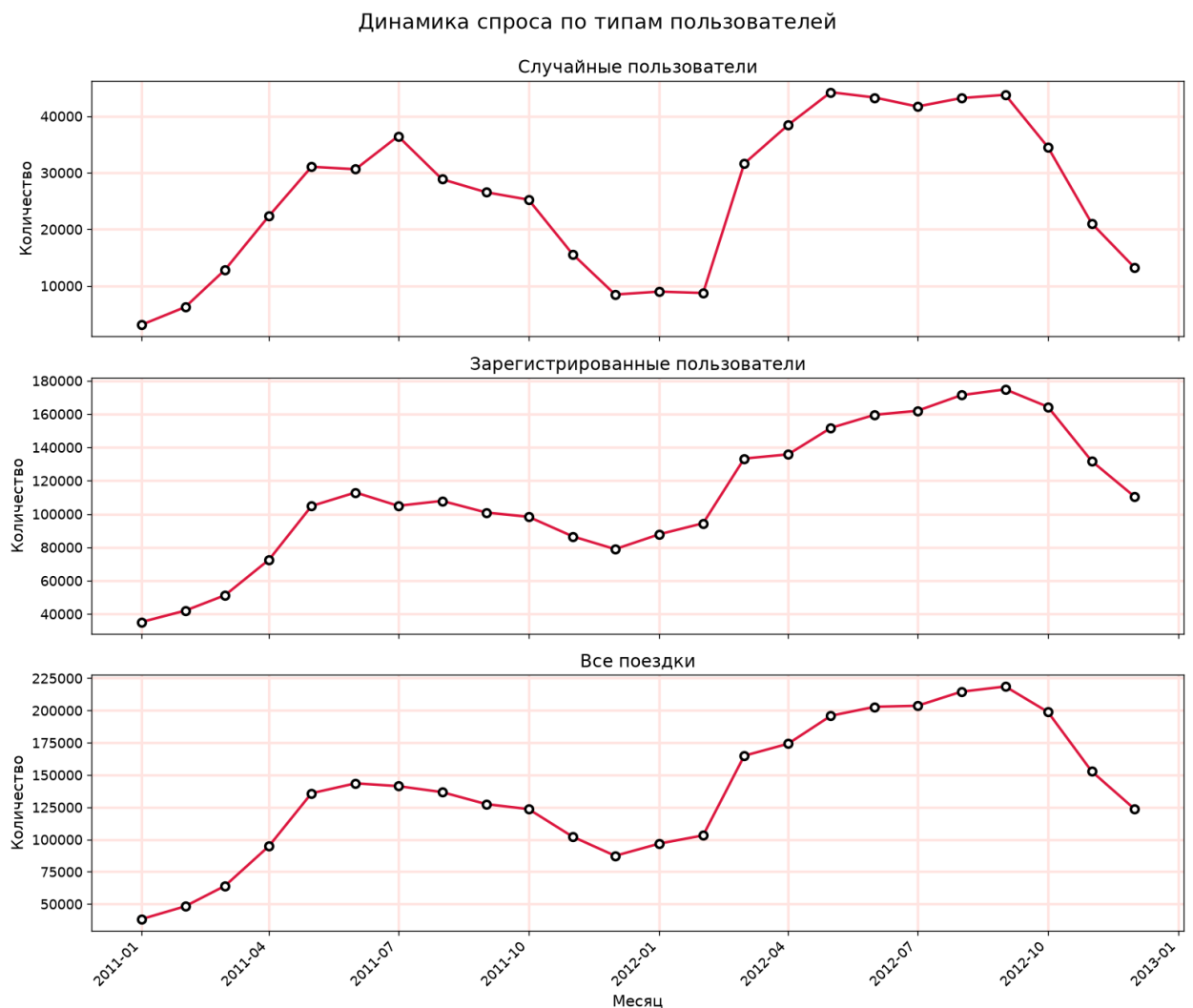


Рисунок 3 – Динамика спроса по типам пользователей

Все три ряда имеют сходную сезонную форму и заметный рост во втором году. Корреляция месячного общего спроса с зарегистрированными пользователями равна 0.991, со случайными — 0.922. Следовательно, основную динамику общего ряда задаёт зарегистрированная аудитория.

7. Визуализация многомерных данных t-SNE

Для снижения размерности использован встроенный набор `load_digits: 1797` изображений цифр 0–9, каждое описано 64 интенсивностями пикселей. Пропуски отсутствуют; признаки стандартизированы. Во всех запусках зафиксирован `random_state=42`, изменялась только перплексия.

```
TSNE(n_components=2, perplexity=perplexity,
     init="pca", learning_rate="auto", max_iter=1000,
     random_state=42).fit_transform(X_scaled)
```

Таблица 4 – Результаты t-SNE

Параметры	Время, с	Trustworthiness@10
perplexity=5	1.4072	0.9807
perplexity=30	1.5196	0.9864
perplexity=50	1.9295	0.9857

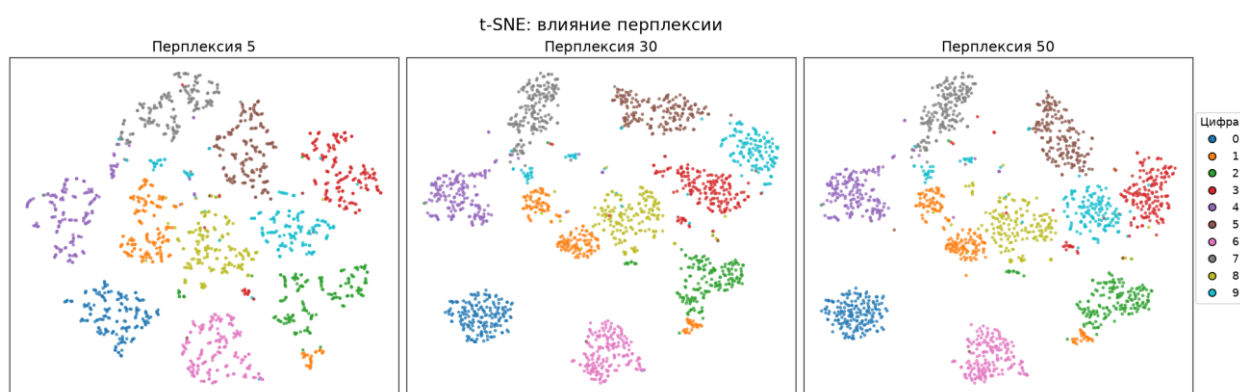


Рисунок 4 – t-SNE при разных значениях перплексии

При перплексии 5 локальные группы сильнее дробятся. Значения 30 и 50 дают более цельные кластеры; лучший измеренный показатель сохранения локальных соседств получен при perplexity=30. Частичные пересечения объяснимы сходным написанием некоторых цифр. Положение далёких кластеров нельзя трактовать как точное расстояние в исходном 64-мерном пространстве.

8. Визуализация многомерных данных UMAP

UMAP применён к тем же стандартизированным объектам. Уменьшение `n_neighbors` усиливает локальные детали, а уменьшение `min_dist` делает группы компактнее. Чтобы исключить многопоточную невоспроизводимость, использованы `random_state=42` и `n_jobs=1`.

```
umap.UMAP(n_components=2, n_neighbors=n_neighbors,
min_dist=min_dist, metric="euclidean",
random_state=42, n_jobs=1).fit_transform(X_scaled)
```

Таблица 5 – Результаты UMAP

Параметры	Время, с	Trustworthiness@10
n_neighbors=5, min_dist=0.0	8.1133	0.9780

Параметры	Время, с	Trustworthiness@10
n_neighbors=15, min_dist=0.1	2.4129	0.9768
n_neighbors=50, min_dist=0.5	2.9469	0.9618

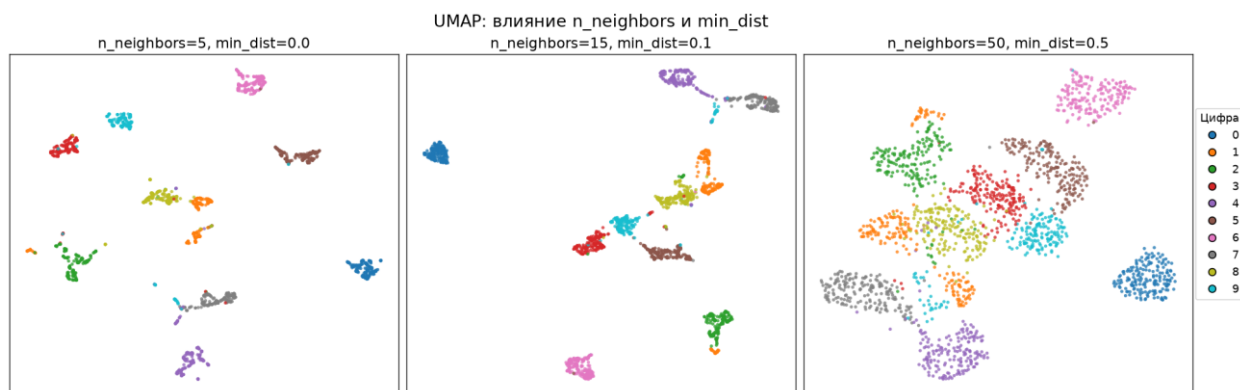


Рисунок 5 – UMAP при разных n_neighbors и min_dist

Конфигурация n_neighbors=5, min_dist=0 формирует самые компактные группы. При n_neighbors=50 и min_dist=0.5 облака становятся шире, а локальная метрика trustworthiness снижается. Поэтому параметры следует выбирать под задачу, а не по визуальной привлекательности одной проекции.

9. Сравнение времени работы

Таблица 6 – Измеренное время всех запусков

Алгоритм	Параметры	Время, с	Trustworthiness@10
t-SNE	perplexity=5	1.4072	0.9807
t-SNE	perplexity=30	1.5196	0.9864
t-SNE	perplexity=50	1.9295	0.9857
UMAP	n_neighbors=5, min_dist=0.0	8.1133	0.9780
UMAP	n_neighbors=15, min_dist=0.1	2.4129	0.9768
UMAP	n_neighbors=50, min_dist=0.5	2.9469	0.9618

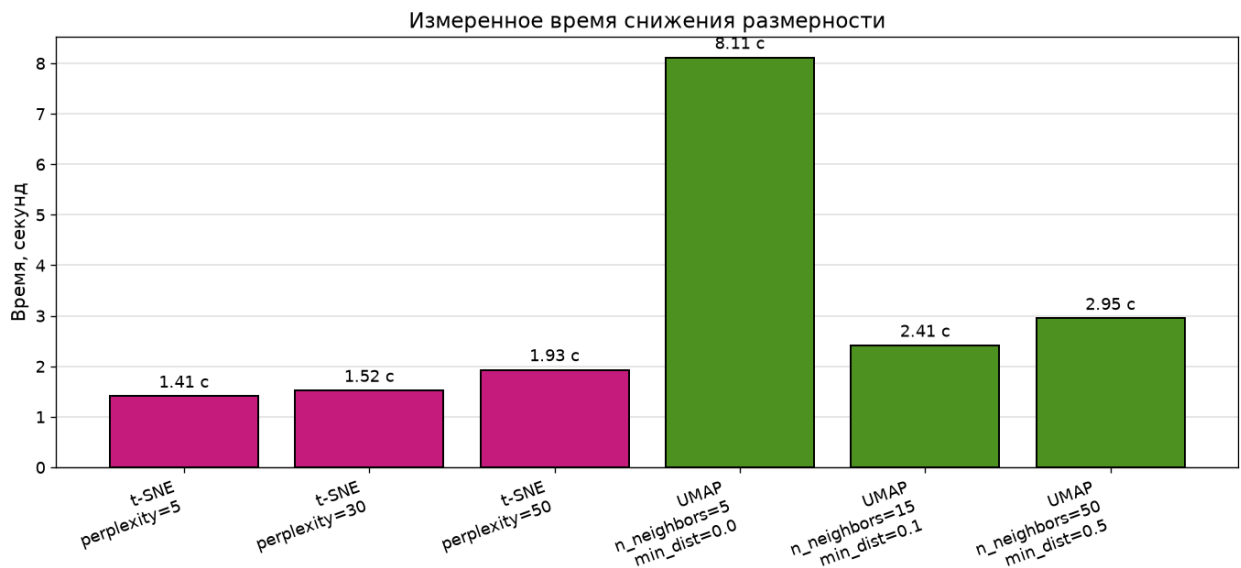


Рисунок 6 – Сравнение времени снижения размерности

Среднее время t-SNE составило 1.6188 с. Среднее время всех запусков UMAP — 4.4910 с, а двух запусков после первого — 2.6799 с. В данном небольшом наборе t-SNE оказался быстрее. Первый запуск UMAP включает JIT-компиляцию Numba, поэтому результат нельзя обобщать на большие наборы и другие аппаратные среды.

10. Сравнение библиотек

Таблица 7 – Сильные стороны и ограничения инструментов

Инструмент	Сильная сторона	Ограничение
Matplotlib	Точный контроль статического рисунка, подграфиков и публикационного экспорта	Интерактивность требует дополнительных средств; много ручных настроек
Plotly	Интерактивные подсказки, масштабирование и автономный HTML	graph_objects многословнее; статический экспорт зависит от браузерного движка
t-SNE	Хорошо подчёркивает локальные соседства и разделение кластеров	Чувствителен к перплексии; глобальные расстояния ненадёжны; нет штатного transform для новых объектов
UMAP	Управляет балансом локальной и глобальной структуры и допускает transform	Результат зависит от параметров; первый запуск несёт расходы JIT-компиляции

11. Итоговые выводы

- Набор Bike Sharing содержит 731 полных дневных наблюдений за два года; пропуски и полные дубликаты отсутствуют.
- За период зарегистрировано 3 292 679 поездок; максимум месячного спроса достигнут в 2012-09.
- Спрос сезонен и во втором году заметно выше; общий месячный ряд теснее связан с зарегистрированной аудиторией.
- Plotly удобен для интерактивного исследования и точных подсказок, Matplotlib — для контролируемой статической композиции.
- t-SNE при perplexity=30 лучше всего сохранил локальные соседства среди проверенных настроек; малое значение сильнее дробит группы.
- UMAP при малых n_neighbors и min_dist создаёт компактные локальные кластеры; рост параметров делает проекцию более разреженной.
- На этом небольшом наборе t-SNE был быстрее UMAP; сравнение времени зависит от JIT-прогрева, объёма данных и окружения.
- Двумерные проекции являются инструментом исследования, а не доказательством истинных кластеров или точных межкластерных расстояний.

12. Ответы на контрольные вопросы

Ниже приведены краткие ответы по объектной модели Matplotlib, возможностям Plotly и интерпретации методов снижения размерности.

1. Из каких основных объектов состоит график Matplotlib?

Figure — весь рисунок, Axes — отдельная область построения, Axis — координатная ось с масштабом, делениями и подписями. На одной Figure может быть несколько Axes.

2. Чем интерфейс pyplot отличается от объектного интерфейса Matplotlib?

pyplot управляет текущим рисунком и областью построения через последовательность команд. Объектный интерфейс изменяет конкретные

объекты `fig` и `ax`, поэтому он удобнее для нескольких подграфиков и точной настройки.

3. Зачем явно вызывать `figure()` или `subplots()`?

Так задаются размер и структура рисунка и исключается неявное изменение ранее активного графика. `subplots()` сразу возвращает объекты рисунка и областей построения.

4. Для чего используется `zorder`?

Параметр определяет порядок наложения объектов. Элемент с большим `zorder` рисуется поверх элемента с меньшим значением.

5. В чём различие `Plotly Express` и `graph_objects`?

`Plotly Express` быстро строит готовый рисунок одной функцией и подходит для типовых задач. `graph_objects` даёт низкоуровневый контроль над следами, макетом, несколькими осями и сложными композициями.

6. Почему `Plotly` называют интерактивной библиотекой?

Рисунок поддерживает наведение, масштабирование, панорамирование, скрытие серий, выбор точек и экспорт. Его можно сохранить как HTML и использовать без запущенного Python.

7. Что показывает цветовая шкала в столбчатой диаграмме этой работы?

Цвет кодирует то же месячное количество поездок, что и высота столбца. Это избыточное, но наглядное кодирование помогает быстрее различать низкие и высокие значения.

8. Почему круговую диаграмму не следует перегружать категориями?

Человек неточно сравнивает близкие углы и площади, а подписи начинают пересекаться. Поэтому мелкие категории объединяют, либо заменяют диаграмму упорядоченными столбцами.

9. Как выбирать между `Matplotlib` и `Plotly`?

`Matplotlib` удобен для статических публикационных рисунков и полного контроля оформления. `Plotly` предпочтителен для интерактивного анализа,

подсказок и веб-представления. Выбор зависит от способа использования результата.

10. Какую задачу решает снижение размерности?

Оно отображает объекты с большим числом признаков в пространстве меньшей размерности, стараясь сохранить выбранные отношения между объектами. Для визуализации обычно используют две координаты.

11. Что означает перплексия в t-SNE?

Это параметр масштаба локального соседства. Малое значение подчёркивает очень локальные группы и может дробить кластеры, большое учитывает более широкое окружение и может сглаживать мелкую структуру.

12. Почему результат t-SNE нужно воспроизводить с `random_state`?

Оптимизация начинается с инициализации и использует стохастические процедуры. Без фиксации генератора разные запуски могут дать различающиеся проекции.

13. Можно ли трактовать расстояния и размеры кластеров t-SNE буквально?

Нет. t-SNE прежде всего сохраняет локальные соседства. Расстояние между далёкими кластерами, их ориентация и видимая площадь не являются надёжными количественными характеристиками исходного пространства.

14. За что отвечают `n_neighbors` и `min_dist` в UMAP?

`n_neighbors` задаёт масштаб соседства: малое значение усиливает локальные детали, большое лучше отражает общую структуру. `min_dist` ограничивает плотность точек в проекции: малое значение делает кластеры компактнее.

15. Почему первое измерение времени UMAP может быть завышено?

Реализация использует Numba и при первом запуске компилирует часть функций. Поэтому корректно отдельно оговаривать «холодный» запуск и сравнивать повторные запуски в одном окружении.

16. Почему двумерная визуализация не доказывает существование истинных кластеров?

Проекция неизбежно теряет информацию и может создавать визуально убедительные группы. Выводы проверяют устойчивостью к параметрам, несколькими метриками и предметной интерпретацией исходных признаков.